

Book Management System Documentation



Figure 1 Library Picture

Table of Contents

List Of Figures	3
List of Abbreviations.....	4
1.0 Project Overview	5
2.0 Technologies Used	5
2.1 Backend Technologies	5
2.2 Frontend Technologies	6
3.0 System Architecture	6
3.1 Runtime Architecture	7
3.1.1 Backend Modules	7
3.1.2 Frontend Structure	8
3.1.3 Database Design	8
4.0 API Endpoints	10
4.1 Authentication Endpoints	11
4.2 Books Endpoints	11
4.3 Users Endpoints	11
4.4 Statistics Endpoints	11
5.0 Authentication Flow	12
5.1 Login Process	12
6.0 Installation Instructions	12
6.1 Backend Installation	13
6.2 Frontend Installation	13
7.0 Setup Instructions	13
7.1 Backend Environment Setup	13
7.2 Run Database Migration	13
7.3 Frontend Environment Setup	13
8.0 Screenshots of the Application	13
9.0 Challenges Faced	18
Conclusion	19

List Of Figures

Figure 1 Library Picture	1
Figure 2 Book Management System Directory	7
Figure 3 Runtime Architecture	7
Figure 4 User Model Schema	8
Figure 5 Book Model Schema	9
Figure 6 Database Table Relationships.....	9
Figure 7 Pgadmin 4 showing users table.....	10
Figure 8 Pgadmin 4 showing books table	10
Figure 9 Landing Page Screenshot	14
Figure 10 Login Page	14
Figure 11 Register Page	15
Figure 12 Users Dashboard	15
Figure 13 Admin Dashboard	16
Figure 14 Book Manangement Section.....	16
Figure 15 Users Manangement Section	17
Figure 16 Profile Page Section.....	17
Figure 17 Swagger UI	18
Figure 18 Source control showing commits made when pushing to Github made using Git Bash.....	18

List of Abbreviations

Abbreviation Full Meaning

API	Application Programming Interface
SPA	Single Page Application
JWT	JSON Web Token
ORM	Object Relational Mapping
DTO	Data Transfer Object
CRUD	Create, Read, Update, Delete
UI	User Interface
UX	User Experience
REST	Representational State Transfer
HTTPS	HyperText Transfer Protocol Secure
CORS	Cross-Origin Resource Sharing
JSON	JavaScript Object Notation
SQL	Structured Query Language
DB	Database
PostgreSQL	Postgre Structured Query Language
npm	Node Package Manager
CLI	Command Line Interface
bcrypt	Blowfish Cryptographic Hash Function

1.0 Project Overview

The **Full-Stack Book Management System** is a web-based application developed to manage a digital catalogue of books with secure authentication, role-based authorization, and administrative management features.

The application allows users to:

- Register and log into the system
- Manage books through create, read, update, and delete operations
- Search books using different criteria
- View dashboard statistics
- Edit personal profile information

Administrators additionally have permission to:

- Manage users
- Delete books
- Access all user records
- Perform restricted administrative operations

The project follows a full-stack architecture using a React frontend and a Nestjs backend connected to a PostgreSQL database through Prisma ORM.

2.0 Technologies Used

2.1 Backend Technologies

Technology	Purpose
Nestjs	Backend REST API framework
TypeScript	Type-safe backend development
PostgreSQL	Relational database
Prisma ORM	Database ORM and schema management
Prisma PostgreSQL Adapter	PostgreSQL driver adapter
bcrypt	Password hashing

Technology	Purpose
Passport JWT	JWT authentication strategy
Nest JWT	Token creation and verification
class-validator	DTO validation
Swagger UI	API documentation

2.2 Frontend Technologies

Technology	Purpose
React	Frontend SPA framework
Vite	Development and build tool
TypeScript	Type-safe frontend development
React Router	Client-side routing
Axios	HTTP communication
React Hook Form	Form management
React Hot Toast	Notifications
Tailwind CSS	Styling
Lucide React	Icons

2.3 Database Technologies

- Type ORM Prisma
- PostgreSQL

3.0 System Architecture

The project follows a monorepo structure:

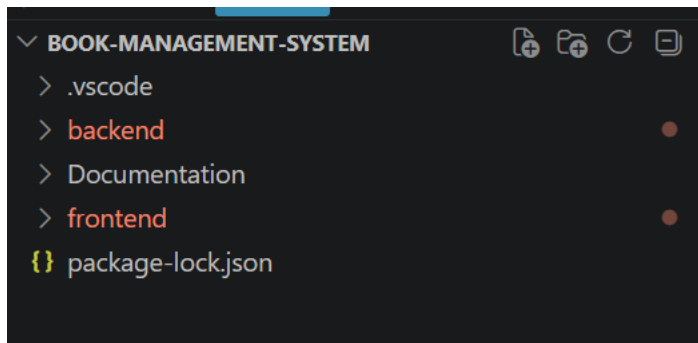


Figure 2 Book Management System Directory

3.1 Runtime Architecture

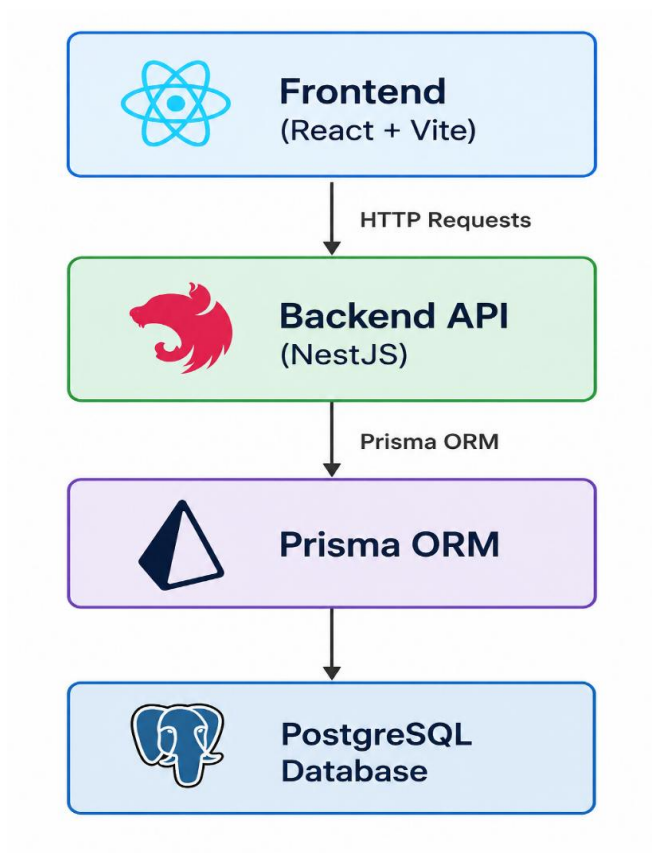


Figure 3 Runtime Architecture

3.1.1 Backend Modules

- Auth Module
- Users Module
- Books Module
- Stats Module
- Prisma Module

3.1.2 Frontend Structure

- Pages
- Components
- Context API
- Services
- Routes

3.1.3 Database Design

The system uses PostgreSQL with Prisma ORM.

User Model

```
model User {
  id          Int      @id @default(autoincrement())
  email       String   @unique
  name        String
  username    String   @unique
  password    String
  role        Role      @default(USER)
  createdAt   DateTime @default(now())

  books Book[]
}

enum Role {
  USER
  ADMIN
}
```

Figure 4 User Model Schema

Book Model

```
model Book {  
  id          Int      @id @default(autoincrement())  
  title       String  
  author      String  
  isbn        String   @unique  
  publishedYear Int  
  description  String  
  createdAt   DateTime @default(now())  
  updatedAt   DateTime @updatedAt  
  
  userId Int  
  user     User @relation(fields: [userId], references: [id])  
}
```

Figure 5 Book Model Schema

Relationship

One User → Many Books

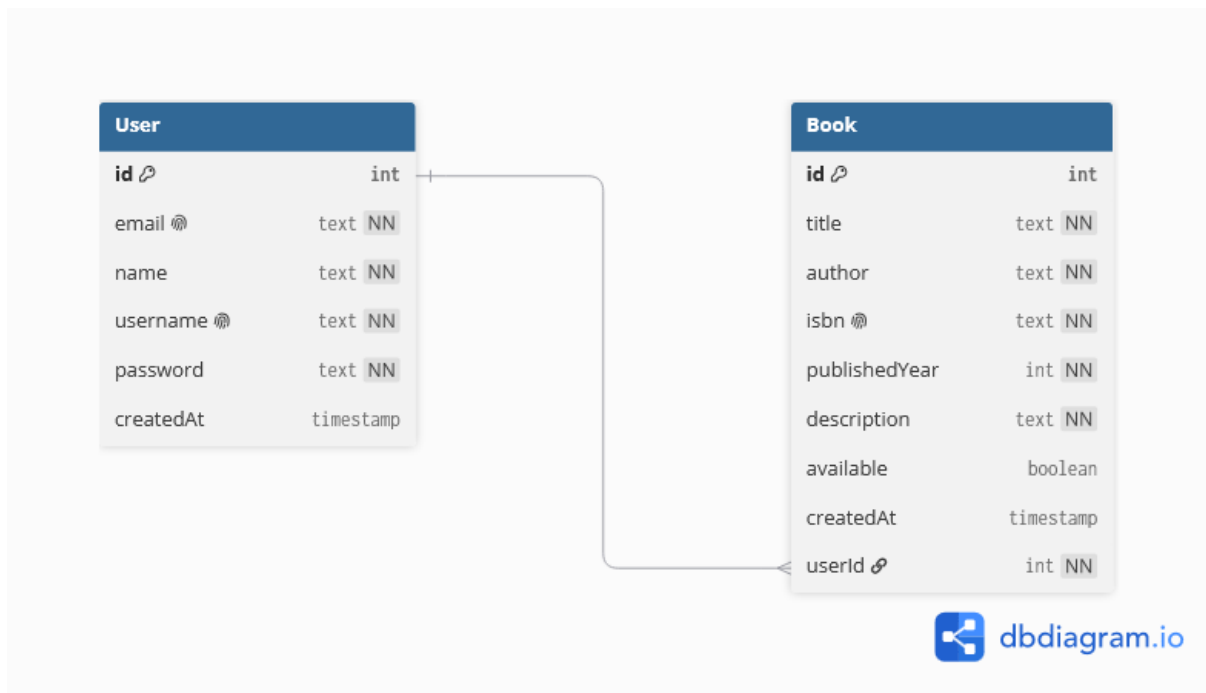


Figure 6 Database Table Relationships

Users Table

pgAdmin 4

public.User/postgres/postgres@Book-Management-System

Query: `SELECT * FROM public."User" ORDER BY id ASC`

Data Output: Showing rows: 1 to 11 of 1

	id	email	name	username	password	createdAt
1	1	test@test.com	Ryan Tselele	Ryie037	\$2b\$10\$G4rlmk9NqilofispGZMG.MUw1chqgy86geVvaJybit5ICScJw6ly	2026-05-23
2	2	admin2026@bookclub.c...	Trevor Nyamadza...	admin01	\$2b\$10\$olhh2QWZyHxk2i26SY8Zouqk8wFXEDuMbwCpLcwrknueoXwIF1u	2026-05-23
3	3	tsele@gmail.com	Craig Tsele	kiri01	\$2b\$10\$gBdJLZHLBb2e2Y.44F075uC6ifm4/kFliHSp9aZbUqcQs/3p.Ap0i	2026-05-24
4	4	testbooks@example.c...	Test	testbooksuser	\$2b\$10\$rmCH6.3a4M/ntA3m81ReOQ2PAwLuOJJyBgE2.mTVHTCVjqaKzV6C	2026-05-24
5	5	testbooks2@example.c...	Test	testbooksuse...	\$2b\$10\$D2iileYAWxdmCm.mPB3W5uA24tAHbcvaP38LkpCDrFWdrqkpnWe6	2026-05-24
6	6	tselecraig@gmail.com	Craig Tsele	Craig34	\$2b\$10\$iluvx7EQMDWZfrq0YHAbOWGDGy4wkI4Bw2C.48Ha0CA7XXJ.RQWC	2026-05-24
7	7	shammah01@gmail.com	Shammah Shamm...	Shamm01	\$2b\$10\$634JLfsy5lmihaPPAHZcpOmZtgSmU0AJkS7xgBkQ3RbyzTGb7diWO	2026-05-26
8	8	dee@gmail.com	dee work	deeman	\$2b\$10\$Rba1sbnfotShPyVwoHnOv/uGmLM6BAV6dudTc/elBZu2M/8vj0D	2026-05-26
9	9	admin@library.com	System Admin	admin	\$2b\$10\$QwsPUEWou0dzknk.Dlg3guv9qulYVelJT4cSqzCiw6N42IRVfpKC	2026-05-26
10	10	kitso@gmail.com	kitso	kiloto	\$2b\$10\$RXCFMehFapy82eBgibe0YuEpH52kczG6cki0Z3FY4dG8Y17CfyGu	2026-05-26
11	11	libho37@gmail.com	Libo	lib37	\$2b\$10\$QyRoP7yJe/uwLMSoav1mZouOKRs0Lw3RoPWf2RaZ8AuCHjMqm/...	2026-05-26

Total rows: 11 Query complete 00:00:02.611

Figure 7 Pgadmin 4 showing users' table

Books Table

pgAdmin 4

postgres/postgres@Book-Management-System

Query: `SELECT * FROM public."Book" ORDER BY id ASC`

Data Output: Showing rows: 1 to 4 of 1

	id	title	author	isbn	publishedYear	description	createdAt
1	4	Hardy Boys	Franklin W.Dix...	978-0-123456-78-4596...	2003	Niagra falls is calling once again	2026
2	5	The Famous Five	Enid Blyton	1971-2004-G1967	2008	Uncle has been kidnapped its up to the gang to rescue ...	2026
3	6	Introduction to pyth...	Andrew Zano	12938457685943	2024	python fundamentals 101	2026
4	7	Applied AI Engineeri...	Ryan Tsele	2026-0502-A1	2026	An Introduction to Applied AI Engineering	2026

Total rows: 4 Query complete 00:00:00.250

Figure 8 Pgadmin 4 showing books table

4.0 API Endpoints

Nestjs Base URL:

http://localhost:3000

4.1 Authentication Endpoints

Method	Endpoint	Description
POST	/auth/register	Register new user
POST	/auth/login	Login user

4.2 Books Endpoints

Method	Endpoint	Description
POST	/Books	Create book
GET	/Books	Get all books
GET	/books/:id	Get single book
PUT	/books/:id	Update book
DELETE	/books/:id	Delete book

4.3 Users Endpoints

Method	Endpoint	Description
POST	/Users	Create user
GET	/Users	Get all users
GET	/users/:id	Get single user
PATCH	/users/:id	Update user
DELETE	/users/:id	Delete user

4.4 Statistics Endpoints

Method	Endpoint	Description
GET	/stats/public	Public statistics
GET	/stats/dashboard	Dashboard statistics

5.0 Authentication Flow

The application uses JWT authentication.

5.1 Login Process

1. User submits email and password
2. Backend validates credentials
3. Password is compared using bcrypt
4. JWT token is generated
5. Token is returned to frontend
6. Frontend stores token in localStorage
7. Token is attached to protected API requests

JWT Payload Example

```
{  
  "sub": 1,  
  "email": "reader@example.com",  
  "role": "USER"  
}
```

Authorization Rules

Action	USER	ADMIN
View Books	Yes	Yes
Create Books	Yes	Yes
Delete Books	No	Yes
Manage Users	No	Yes

6.0 Installation Instructions

Prerequisites

- **Node.js 18+**
- **npm**
- **PostgreSQL**

6.1 Backend Installation

cd backend

npm install

6.2 Frontend Installation

cd frontend

npm install

7.0 Setup Instructions

7.1 Backend Environment Setup

Create .env inside the backend folder:

DATABASE_URL="postgresql://postgres:password@localhost:5432/postgres?schema=public"

JWT_SECRET="your-secret-key"

PORT=3000

7.2 Run Database Migration

npx Prisma migrate dev

Start Backend

npm run start:dev

7.3 Frontend Environment Setup

Create .env inside frontend:

VITE_API_URL=http://localhost:3000

Start Frontend

npm run dev

8.0 Screenshots of the Application

Landing Page

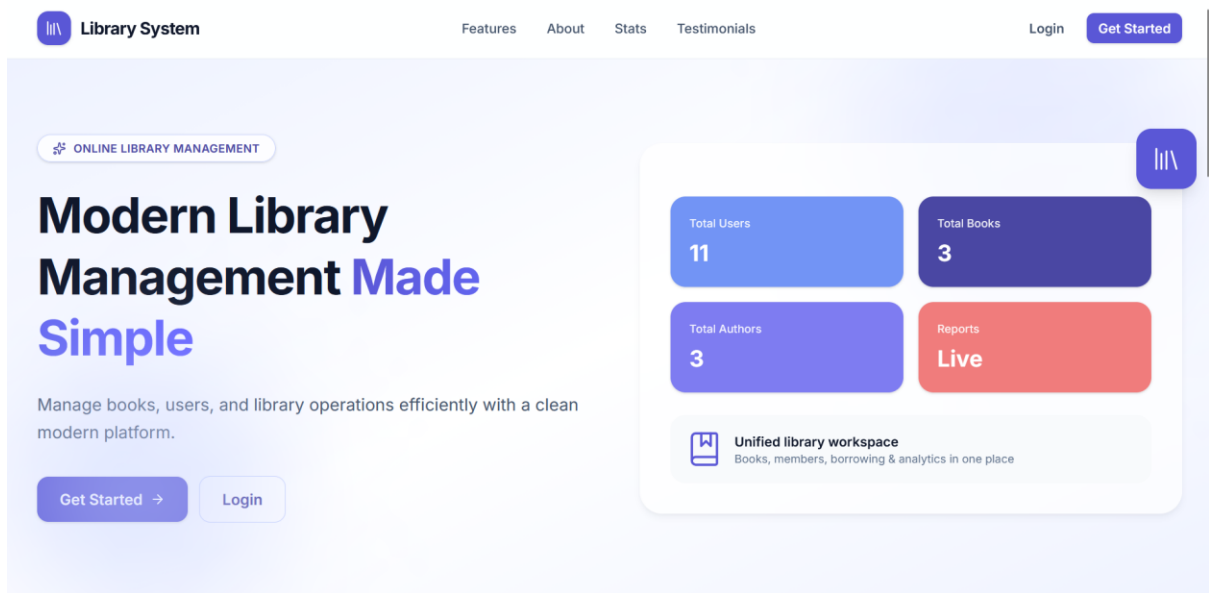


Figure 9 Landing Page Screenshot

Login Page Screenshot

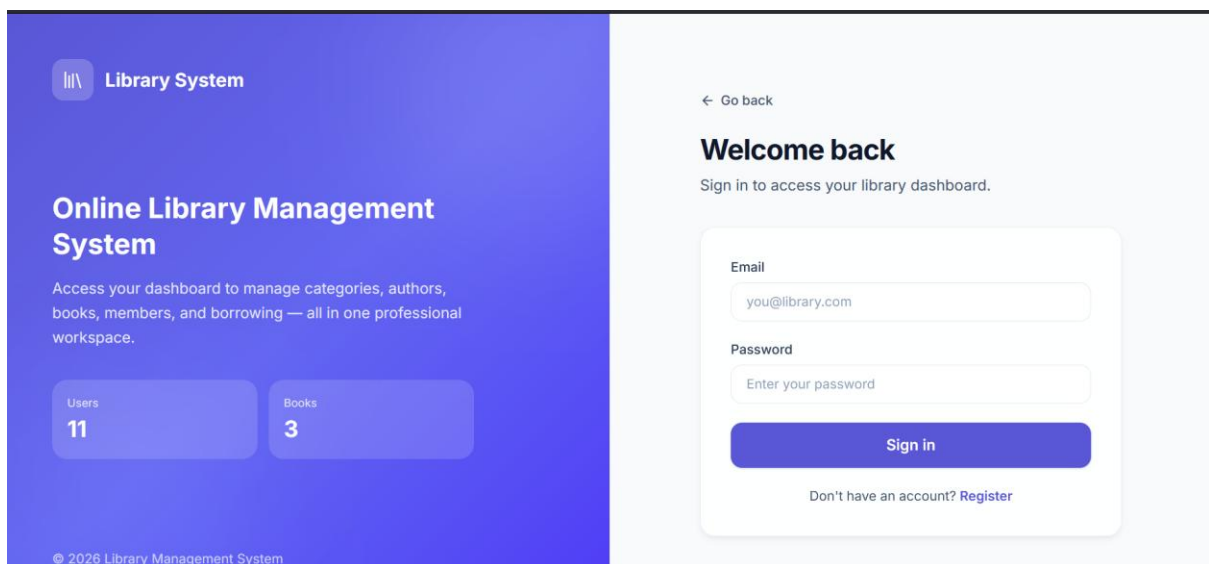


Figure 10 Login Page

Registration Page Screenshot

Library System

Online Library Management System

Access your dashboard to manage categories, authors, books, members, and borrowing — all in one professional workspace.

Users
11

Books
3

[← Go back](#)

Create your account

Join the platform and start managing your library.

Full name

Jane Librarian

Username

janedoe

Email

you@library.com

Password

At least 6 characters

Confirm password

Re-enter your password

Figure 11 Register Page

Dashboard Overview Screenshot

Users Dashboard

Library System

Dashboard

Books

Profile

[Logout](#)

Online Library Management System

Dashboard

Overview of your library

Total Books
3

Recent Books
3

Active Authors
3

Quick Actions

Manage books

Manage my profile

Figure 12 Users Dashboard

Admin Dashboard

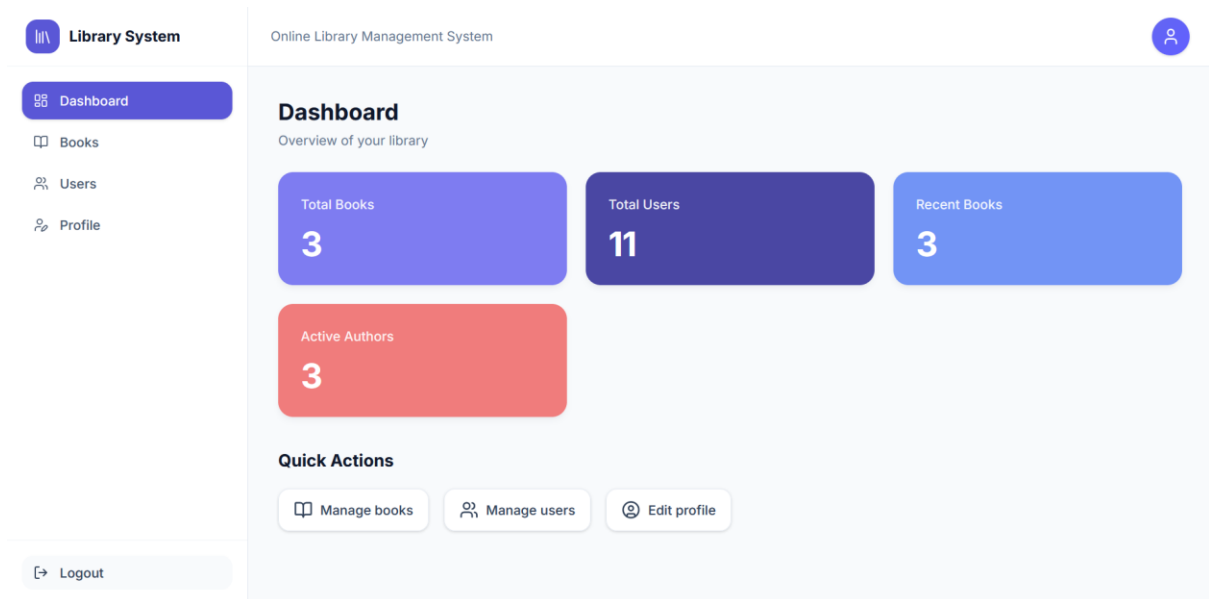


Figure 13 Admin Dashboard

Books Management Page Screenshot

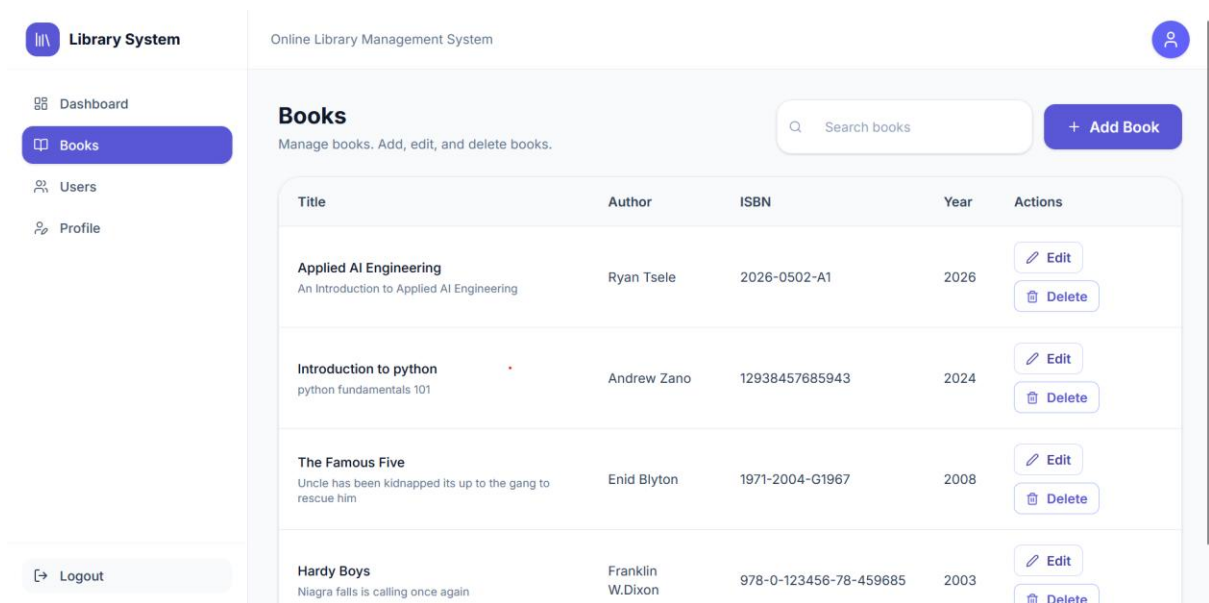


Figure 14 Book Management Section

Users Management Page Screenshot

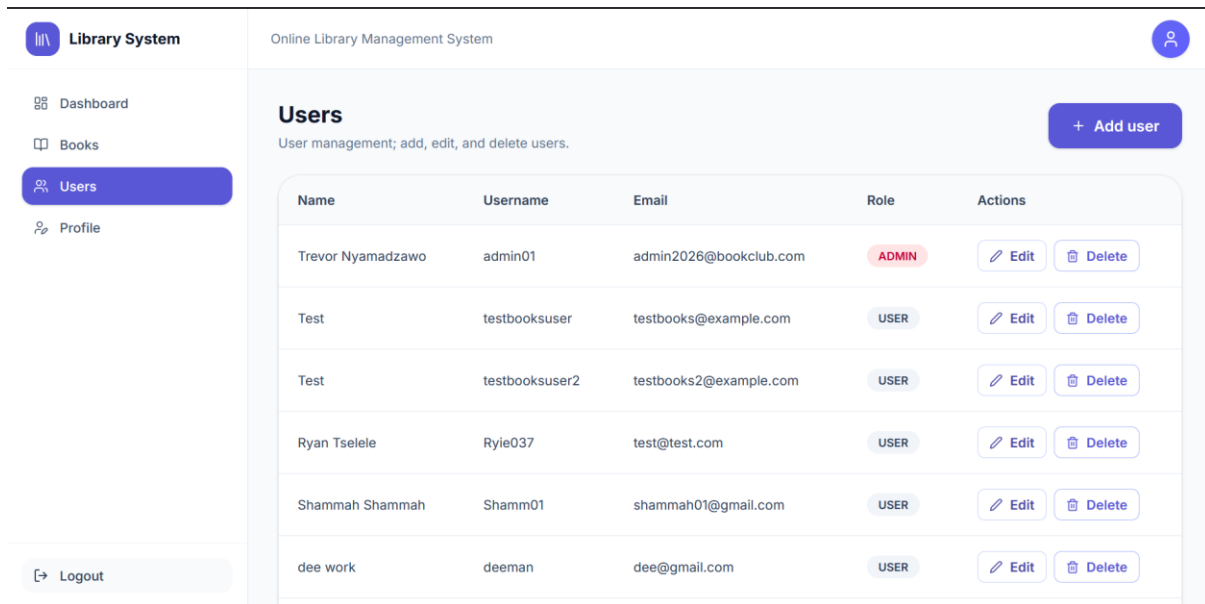


Figure 15 Users Management Section

Profile Page Screenshot

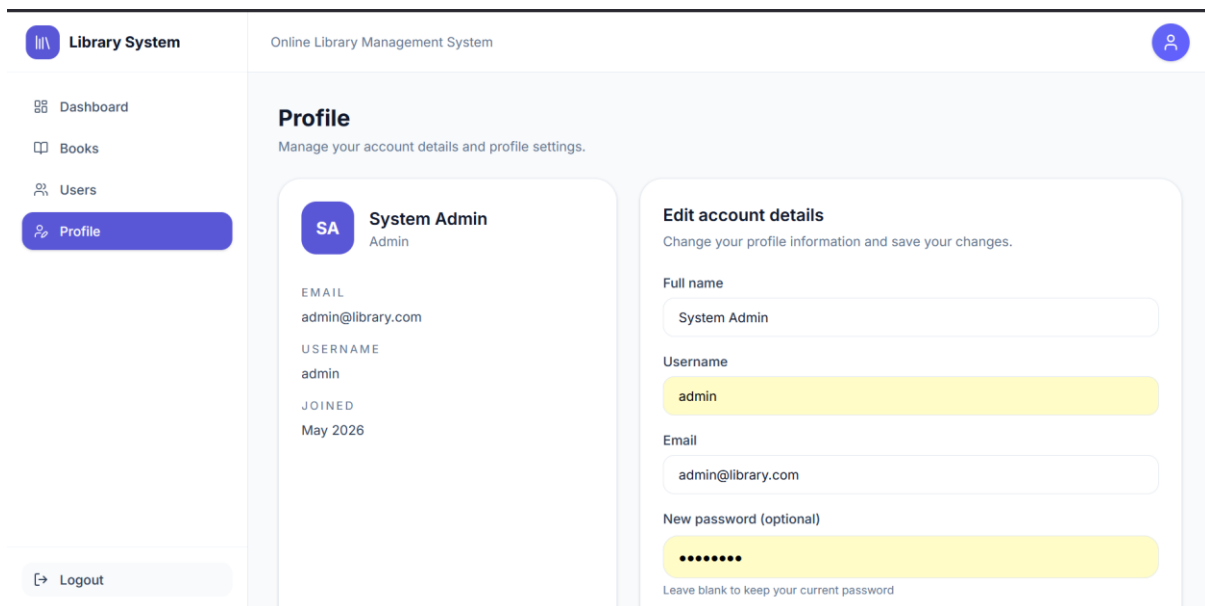


Figure 16 Profile Page Section

Swagger API Documentation Screenshot

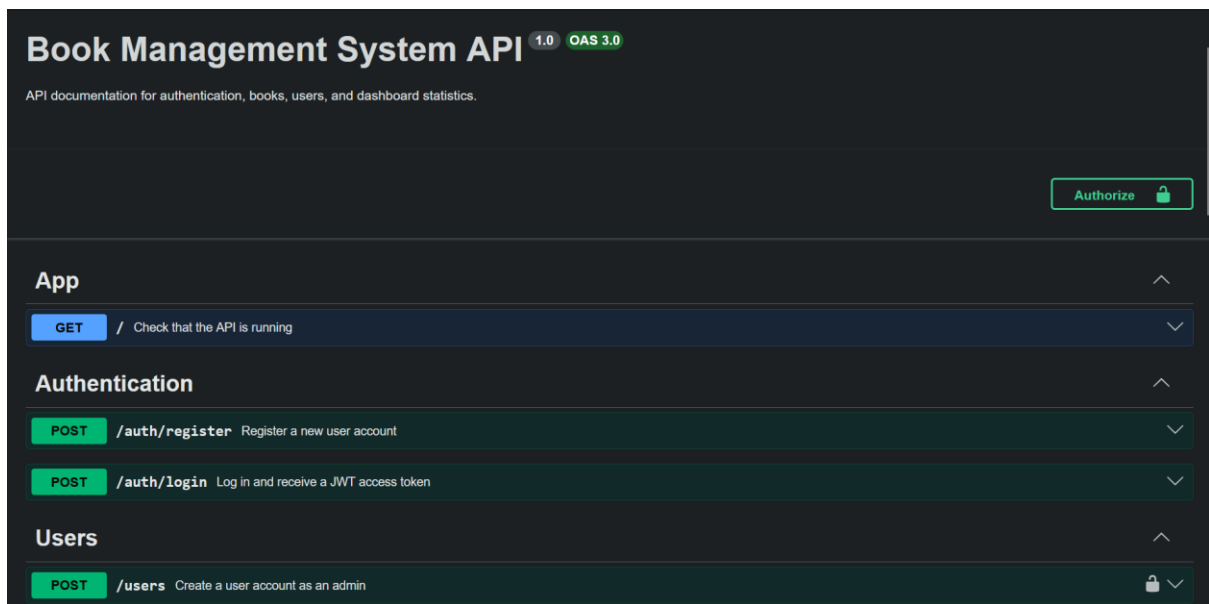


Figure 17 Swagger UI

Source Control view Screenshot

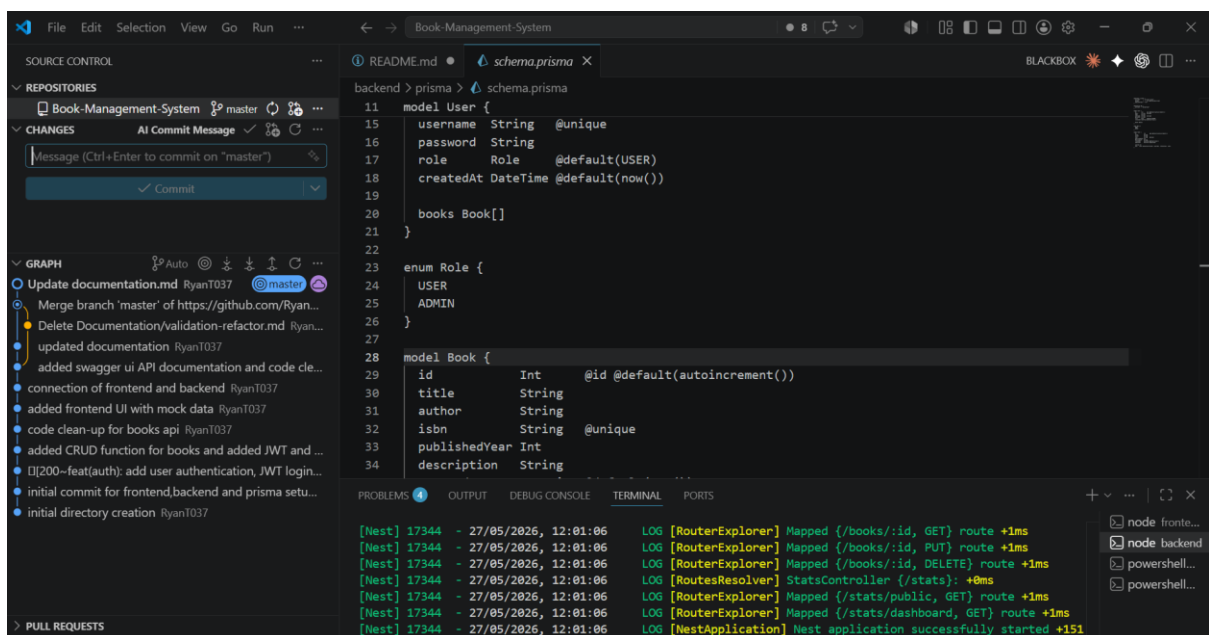


Figure 18 Source control showing commits made when pushing to GitHub made using Git Bash

9.0 Challenges Faced

Prisma PostgreSQL Adapter Setup

Configuring Prisma 7 with the PostgreSQL adapter required custom Prisma client initialization and generated client configuration which slightly took time due to wrong file directories and prettier configurations.

JWT Authentication Configuration

Managing JWT secrets securely and ensuring proper token validation required configuring ConfigService and Passport JWT strategies correctly.

Role-Based Authorization

Implementing owner/admin authorization logic for books and admin-only access for users required careful backend validation and controller protection.

Frontend Token Persistence

Managing JWT storage and automatic Axios authorization headers required implementing interceptors and authentication context handling.

Real-Time Statistics

Dashboard and landing page statistics required aggregation queries using Prisma ORM.

Conclusion

The Full-Stack Book Management System successfully demonstrates a modern full-stack web application architecture using React, Nestjs, PostgreSQL, and Prisma ORM.

The system includes:

- Secure JWT authentication
- Role-based access control
- CRUD functionality
- Dashboard statistics
- REST API architecture
- Frontend and backend integration

The project provides a scalable foundation for future enhancements such as pagination, deployment, automated testing, and advanced analytics.